



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance measure
E - Environment
A - Actuators
S - Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.food_positions  
self.opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Baseline environment is classified as "Multi-Agent".

The environment is Multi-Agent because it contains more than one agent interacting within the same environment.

In the code, `self.agent_pos = [0, 0]` represents the main agent and `self.opponents = []` stores one or more opponent agents that are generated using `num_opponents` parameter. During execution each opponent moves independently and directly impacting the main agent's state by collisions and reducing the score. Therefore, since the main agent and the opponent agents coexist and influence each other's outcomes, the environment should be classified as a Multi-Agent Environment

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of all perceptions an agent has received over time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensor component

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what

the agent can and cannot see.

Partially Observable.

A fully observable environment requires the agent's sensors to provide access to the complete state of the environment at all times. However, the data returned by `get_percept()` is limited.

what the agent can see :- `agent_pos` (has global knowledge of its own exact location), `opponent_positions` (the exact locations of all enemies), `score` and `collision` (the current game state), and `remaining_food` (the total count of food items left).

what the agent cannot see :- The exact locations of all food items (`self.food_positions`), the exact locations of all walls (`self.walls`), the complete layout of the grid. because the agent is blind to the global layout of the food and obstacles and must discover them through exploration, the agent must act without complete knowledge of the world.

**Task 1.3: Action Execution (`execute_action`)**

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure component



8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because a true Performance measure must evaluate what the agent actually achieves in the environment (the outcome) not how it achieves it (the internal computation).

In this game, hitting a wall is an external environmental event caused by the agent's action and deducting points measures whether the agent is making good decisions in navigating the environment.

If the metric evaluated internal processing effort (such as the number of calculations, memory usage, or decision time), it would not measure the actual goal of the agent, which is to successfully move through the grid, avoid obstacles, collect food, and avoid collisions.



Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If we add `self.toxic_traps` to the environment but hide them from the agent's sensors, the environment becomes even more partially observable. Because the agent already cannot see everything, and if this is hidden too, the agent has even less information about the environment and must guess where traps might be, exploring carefully, and remembering where it has found traps before.



Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.
10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

By adding the 'smells_toxin' sensor, improves the agent's rational decision-making because it provides additional information about potential negative outcomes in the environment. Without this sensor, the agent may randomly move onto a toxic trap and loose points without knowing the danger. The agent's goal (utility) is to achieve the highest possible score. The 'smells_toxin' sensor allows the agent to immediately move away from the current tile and avoid returning to it in the future, by avoiding further point loss and keeping its expected utility high.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.
11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

"Vacuum World Exploit" example illustrates what happens when a performance metric rewards an agent's actions rather than the actual desired outcome. In the example, a vacuum-cleaning agent was rewarded simply for removing dirt, but the performance measure did not properly consider efficiency or cleanliness over time. A faulty agent could exploit this by:

repeatedly cleaning the same cleaned area to gain rewards, moving around unnecessarily to trigger more scoring opportunities, ignoring other important areas because the metric only rewarded a specific action.

Performance measure should be based on the desired state of the environment, not on the agent's behavior. To fix the exploit, the metric must change to reward the environment's state.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING